Программирование

Конспект лекций

Содержание

1 Арифметические выражения **3**

1.1 Приоритет операций . . . 3

2 Линейные алгоритмы **3**

2.1 Основные блоки схемы алгоритма . . . 3

2.2 Правила составления схем . . . 4

2.3 Подпрограммы . . . 4

3 Применение функций пользователя **5**

3.1 Правила оформления схем алгоритмов программ с п/п . . . 6

4 Условный оператор **7**

4.1 Схема условного оператора . . . 7

4.2 Логические выражения . . . 8

4.3 Тернарный условный оператор . . . 8

5 Многоальтернативное ветвление **11**

5.1 Оператор выбора `match` . . . 12

6 Циклы с параметром (со счётчиком) `for` **14**

6.1 Диапазон `range()` . . . 14

6.2 Форматный вывод: оператор `%` . . . 16

6.3 Накопление суммы . . . 17

6.4 Произведение . . . 17

6.5 Табулирование . . . 19

7 Цикл с предусловием `while` **19**

7.1 Цикл с постусловием . . . 20

7.2 Общее для циклов `for` и `while` . . . 20

8 Вложенные циклы **22**

9 Списки (одномерные массивы) **24**

9.1 Перебор элементов списка . . . 26

9.2 Ввод списка с клавиатуры . . . 26

9.3 Вывод элементов списка . . . 26

9.4 Срезы списков . . . 26

9.5 Генератор списков . . . 27

9.6 Функция `enumerate()` . . . 27

9.7 Методы списков 27

10 Матрицы (двумерные массивы) 33

10.1 Создание и ввод матрицы 33

10.2 Вывод матрицы 34

10.3 Пример 34

11 Кортежи 36

11.1 Присваивание кортежей 37

11.2 Для чего нужны кортежи 37

11.3 Ввод кортежа с клавиатуры 38

1. Арифметические выражения

Операция: `17 + 2`. *Оператор* (команда, инструкция).

Арифметическое выражение может состоять из:

1) **операций**:

`+`	сложение	`//`	целочисленное деление
`-`	вычитание	`%`	остаток от деления
`*`	умножение	`**`	возведение в степень
`/`	деление		

2) **скобок**: $\frac{a+b}{c+d}$ $\Rightarrow$ `(a + b) / (c + d)`;
3) **переменных**;
4) **числовых констант**;
5) **математических функций**:

1 способ:	2 способ:
`import math`	`from math import *`
`math.log(4.8)`	`log(4.8)`

6) $\sqrt{x}$ — `sqrt(x)`; `x ** (1/2)` — **комплексное число!** (если ввели отрицательное число);
7) **тригонометрических функций** — аргумент в радианах;
8) `degrees` — перевод из радиан в градусы; `radians` — перевод x из градусов в радианы.

1.1. Приоритет операций

1) то, что в скобках `( )`;
2) математические функции;
3–4) операция возведения в степень; унарная операция `(-)`: `-5`, `-sin(x)`, `+5` (в мат. — «комплементарная»);
5) `* / % //`;
6) `+ -`.

`x = 1 + 2 + 3` — по очереди слева направо.

`2 ** 3 ** 2` → `512` — справа налево: $2^{(3^2)} = 2^9$.

`#` — однострочный комментарий; `'''` многострочный комментарий `'''`.

2. Линейные алгоритмы

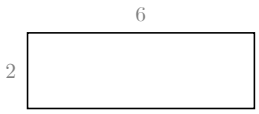
> **Определение.** *Алгоритм* — это конечная последовательность действий решения какой-либо задачи.

Способы представления алгоритма:

1. структурограммы;
2. схема алгоритма;
3. псевдокод;
4. словесная запись.

2.1. Основные блоки схемы алгоритма

Размеры даны в клетках: обычный блок — 6 клеток в ширину и 2 в высоту.



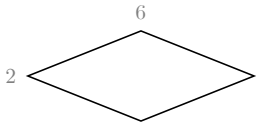
1) вычислительный процесс



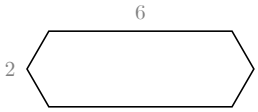
2) начало и конец процесса



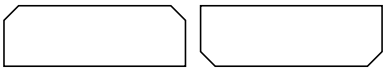
3) ввод и вывод



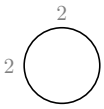
4) решение



5) цикл с параметром (старый ГОСТ)



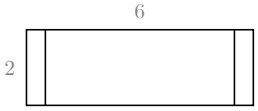
6) цикл с параметром (новый ГОСТ): начало и конец цикла



7) соединитель



8) межстраничный соединитель



9) предопределённый процесс

2.2. Правила составления схем

1) Одинаковая ширина блоков — 6–8 клеток.
2) Высота блоков может быть различной.
3) Соединительная стрелка между блоками — 1 клетка, по необходимости 2.
4) Математические функции пишем как в математике.
5) Подряд идущие ввод данных, вывод данных, присваивания объединяем в один блок.
6) Каждое присваивание в блоке пишем с новой строки.
7) Схема алгоритма изображается без привязки к конкретному языку программирования.
8) Схема алгоритма должна занимать в высоту не более листа А4.

2.3. Подпрограммы

В программировании часто встречаются алгоритмы, в которых повторяются фрагменты, одинаковые по выполняемым действиям и различающиеся только значениями обрабатываемых данных.

Определение. Такие фрагменты записывают отдельно, однократно и называют *подпрограммами* (п/п). В дальнейшем к п/п можно обращаться многократно с различными значениями данных.

Синтаксис описания п/п (`[ ]` — часть, которая может отсутствовать):

```
def <имя п/п>([<список формальных параметров>]):
    <команды>
    [return <возвращаемые значения>]
```

Синтаксис вызова п/п:

```
<имя п/п>([<список фактических параметров>])
```

Определение. *Формальные параметры* — это имена переменных, которые используются для описания п/п.
Фактические параметры — это конкретные значения параметров, с которыми п/п будет вызвана и выполнена.

П/п бывают двух видов: (1) функции; (2) процедуры.

Определение. *Функцией* будем считать п/п, в которой присутствует `return` и возвращаемое значение. П/п без `return` — *процедура*.

Пример. Процедура без параметров:

```
def Error():
    print('Ошибка!')

a = 5
b = 0
if b == 0:
    Error()
else:
    print(a / b)
```

Пример. Процедура с параметрами:

```
def hello(name):
    print('Привет,', name, '!')

name = input('Вв. своё имя: ')
hello(name)
hello('Мама')
hello(1)
```

Пример. Функция сложения:

```
def func(a, b):
    rez = a + b
    return rez

z = func(6, 4)
print(z)            # 10
```

Процедура и функция различаются при вызове. В место вызова функции возвращается значение, которое можно присваивать, выводить, использовать в других выражениях и т. д.

Правила соответствия между формальными и фактическими параметрами:

1. В классическом случае число формальных и фактических параметров должно совпадать.
2. Типы параметров (всех: и формальных, и фактических) в общем случае должны совпадать.
3. Порядок параметров (всех) нужно соблюдать.
4. Имена формальных и фактических параметров могут совпадать.

Определение. *Глобальная переменная* — это переменная, область видимости которой — вся программа. В Питоне это переменные, созданные в основной программе.
Локальная переменная — это переменная, область видимости которой чем-то ограничена (например, телом п/п).

3. Применение функций пользователя

Определение. *Необязательные параметры.* При описании п/п у формальных параметров можно указать их значения по умолчанию. Тогда эти параметры при вызове можно не указывать, т. е. они являются необязательными. Параметры, у которых значения по умолчанию нет, будут обязательными.

```
def proc(arg1, arg2=5, arg3=88):
    print(arg1, arg2, arg3)

proc(1)              # 1 5 88
```

```
proc(1, 2)            # 1 2 88
proc(888, arg3=77)    # 888 5 77
```

Пример 3.1. Программа вычисления суммы площадей двух прямоугольников[1].

Сразу после заголовка функции в тройных кавычках пишут её описание: 1) что делает; 2) перечисляют все входные значения; 3) описывают, что возвращает `return`.

```
# Программа вычисления суммы площадей двух прямоугольников
# Переменные:
#     a1, b1, a2, b2 - стороны прямоугольников
#     S1, S2 - их площади

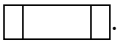
# import ...  (сначала подключаются модули)

def sq(a, b):
    ''' Функция вычисления площади прямоугольника S
    Входные параметры:
        a, b - стороны прямоугольника
    Выходные данные: S
    '''
    S = a * b
    return S            # или сразу: return a * b

print('Вв. стороны 1-го прям.')
a1 = float(input())
b1 = float(input())
print('Вв. стороны 2-го прям.')
a2 = float(input())
b2 = float(input())
# эхо-печать
print(a1, b1, a2, b2)

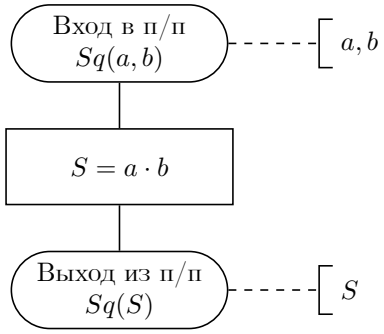
S1 = sq(a1, b1)
S2 = sq(a2, b2)
print('Пл. 1-го прям. =', S1)
print('Пл. 2-го прям. =', S2)
print('Их сумма', S1 + S2)
```

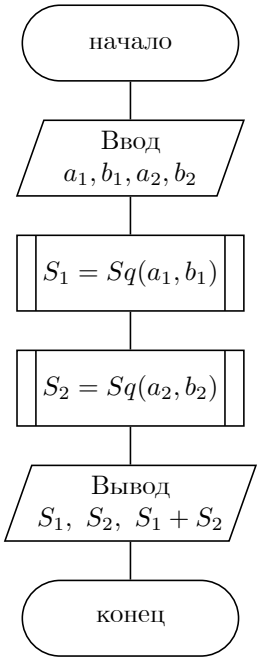
3.1. Правила оформления схем алгоритмов программ с п/п

1) Начало и конец пишут только в основной программе. В п/п пишут *вход* и *выход*.
2) Для каждой п/п изображается отдельная схема алгоритма.
3) В верхнем блоке п/п нужно указать входные параметры, а в нижнем — выходные параметры или вычисляемые значения.
4) Вызов п/п в основной программе изображается блоком «предопределённый процесс»  .

[1] *Исправлено.* В рукописи стороны второго прямоугольника тоже вводятся в `a1`, `b1`; должно быть `a2`, `b2` — иначе `sq(a2, b2)` даст ошибку.

Схемы к примеру 3.1: слева — п/п `Sq`, справа — основная программа.





4. Условный оператор

Условный оператор позволяет выполнить один из входящих в него наборов команд.

Синтаксис:

```
if <выражение>:
    <команда 1>
[else:
    <команда 2>]
```

Сначала проверяется значение выражения после `if`. Если оно истинно, то выполнится `<команда 1>`, иначе (значение выражения ложно) выполнится `<команда 2>` (если есть `else`).

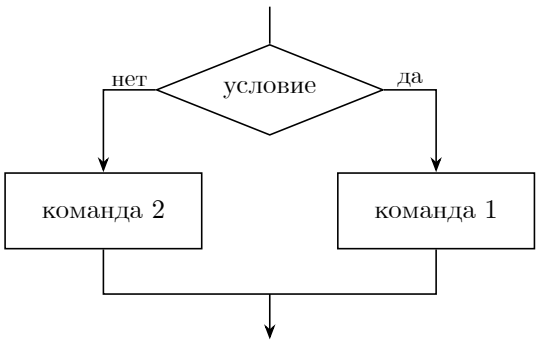
За истину в Питоне считается:

- `True`;
- числовое значение, отличное от нуля;
- непустые объекты.

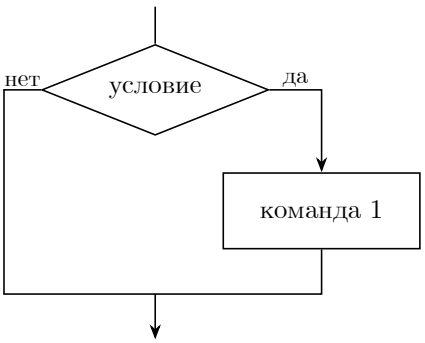
Ложь:

- `False`;
- `0` (целое, вещественное `0.0`);
- пустые объекты (`None`).

4.1. Схема условного оператора

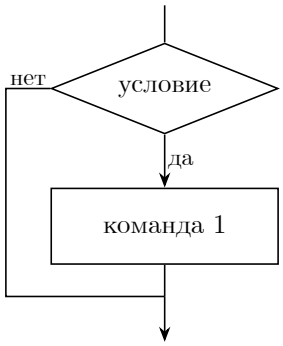


полная форма



сокращённая форма

Условный оператор с одним условием (сокращённая форма) можно изобразить и так:



4.2. Логические выражения

В классическом варианте выражение после `if` должно быть *логическим* (истина или ложь). Чтобы выражение было логическим, оно должно включать:

- **операции отношения (сравнения)**: `>`, `<`, `>=`, `<=`, `==`, `!=`;
- **логические операции**:
  `and` — логическое «и» (конъюнкция);
  `or` — логическое «или» (дизъюнкция);
  `not` — логическое «не» (отрицание).

«Исключающее или» — побитовое сложение по модулю 2, обозначается `^`. Истина, когда аргументы имеют противоположные значения; ложь — когда одинаковые.

! В Питоне у операций сравнения приоритет выше, чем у логических операций.

Пример. $1 \le x \le 3$.
Классический вариант: `x >= 1 and x <= 3`.
В Питоне реализованы «ленивые» цепочки операций сравнения: `1 <= x <= 3`. Сначала вычисляется `1 <= x`. Если это `False`, то дальше не идём (результат `False`). Если `True`, то берём `x` и дальше сравниваем: `x <= 3`.

4.3. Тернарный условный оператор

Конструкцию

```
if X:
    A = Y
else:
    A = Z
```

можно записать в одну строку[2]:

```
A = Y if X else Z
```

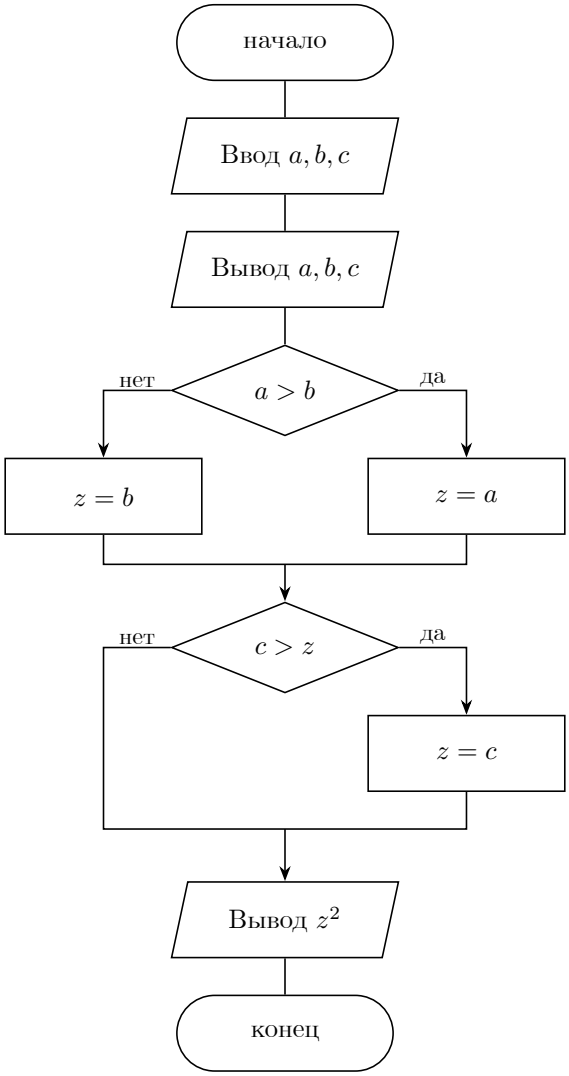
Пример. Найти квадрат максимума из трёх значений.

Переменные: `a, b, c` — исходные числа; `z` — максимум из них.

```
print('введите 3 числа')
a = float(input())
b = float(input())
c = float(input())
# эхо-печать
print(a, b, c)
if a > b:
    z = a
```

[2] *Дописано.* В рукописи под заголовком записана только полная форма `if`–`else`; строка с тернарным оператором добавлена.

```
else:
    z = b
if c > z:
    z = c
print("Квадрат макс =", z * z)
```



Пример 4.2. (к лаб. 4.1). Вычисление выражения с условием:

$$y = \begin{cases} \frac{1}{2}\sqrt{x}, & x \geq 1, \\ \frac{1}{3}\sqrt[3]{x}, & 0 < x < 1, \\ \frac{1}{4}\sqrt[4]{|x|}, & -1 \leq x \leq 0, \\ 0 & \text{иначе.} \end{cases}$$

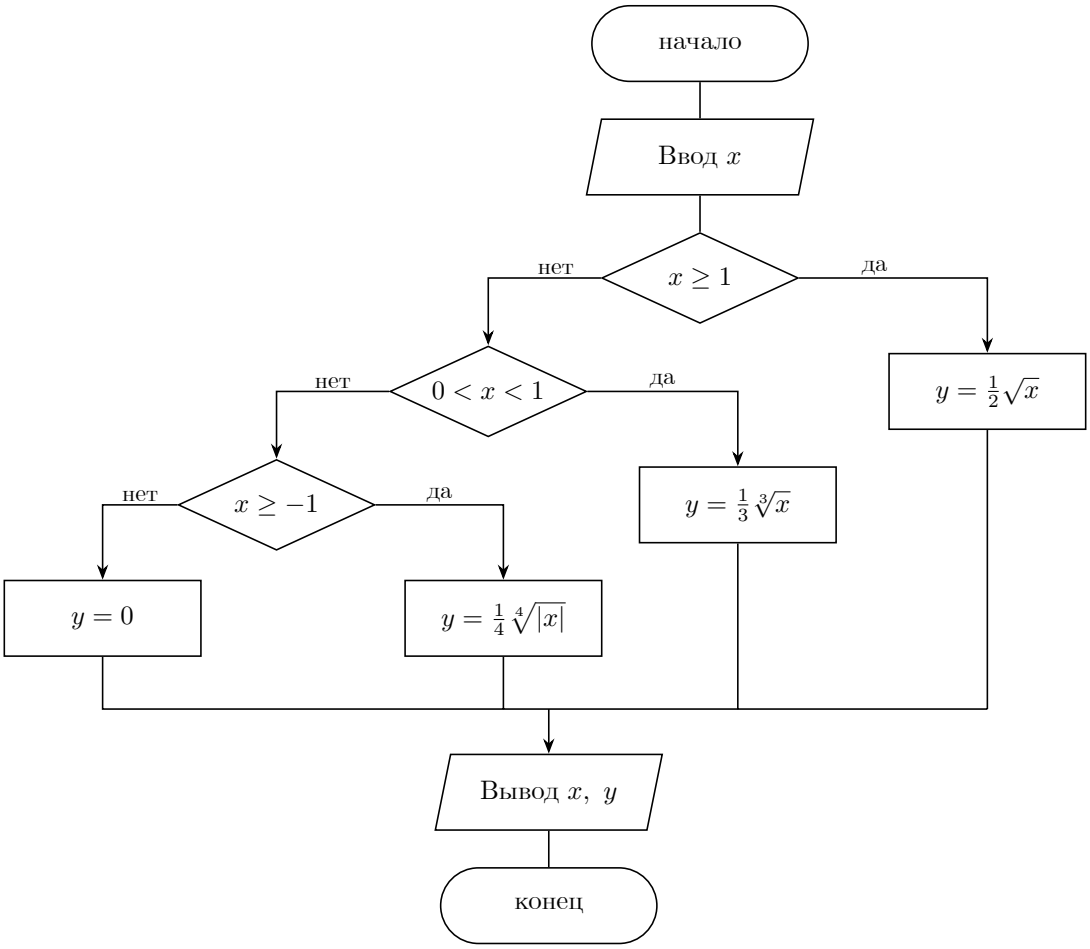
```
# Вычисление выражения с условием
# Переменные:
#     x - аргумент
#     y - вычисляемая функция
from math import sqrt

x = float(input('введите x \n'))

if x >= 1:
    y = 0.5 * sqrt(x)
else:
```

```
    if x > 0 and x < 1:
        y = 1/3 * x ** (1/3)
    else:
        if x >= -1 and x <= 0:
            y = 1/4 * abs(x) ** (1/4)
        else:
            y = 0

print('x =', x, 'y =', y)
```

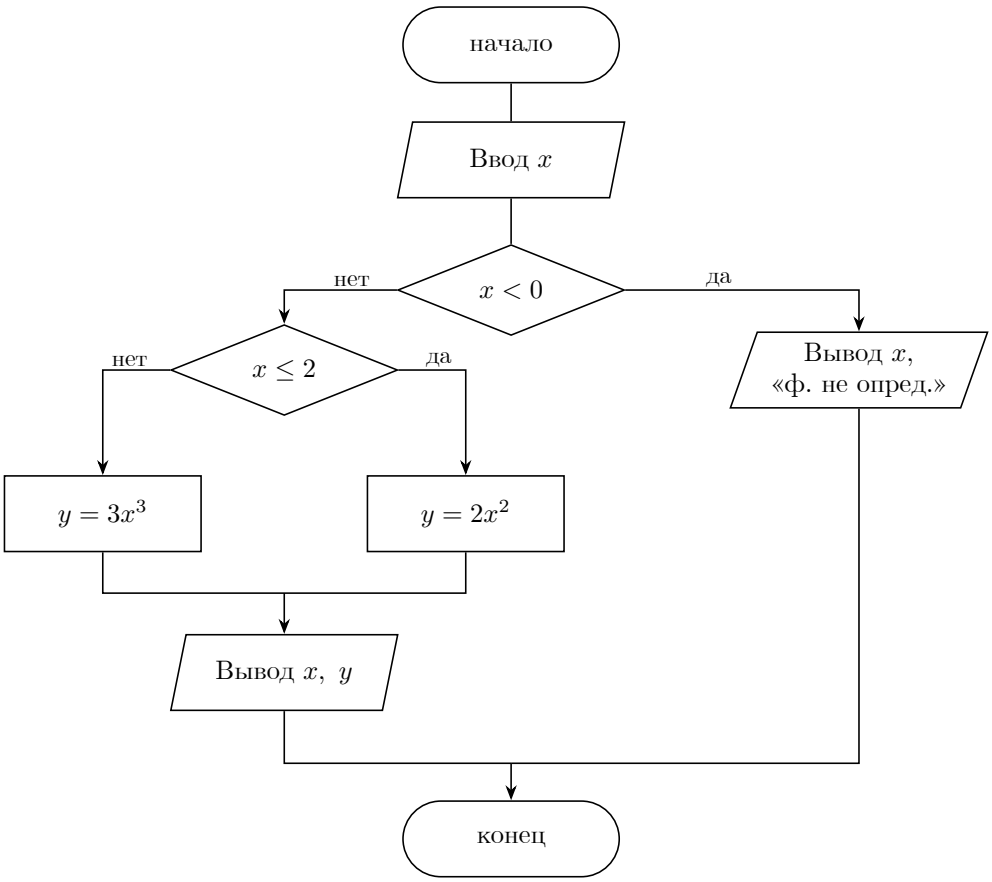


Замечание. Если функция где-то не определена, нужно разделить числовую прямую на части и для каждой части записать свою ветку.

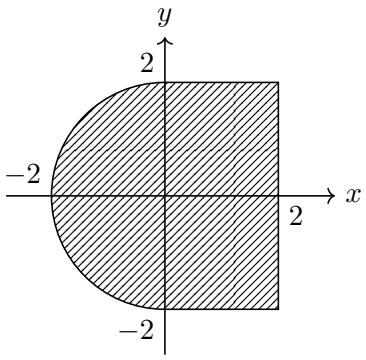
Пример 4.3. $y = \begin{cases} 2x^2, & 0 \leq x \leq 2, \\ 3x^3, & x > 2; \end{cases}$ при $x < 0$ функция не определена[3].

```
# пример 4_3
x = float(input())
if x < 0:
    print('x =', x, 'ф. не определена')
else:
    if x <= 2:
        y = 2 * x ** 2
    else:
        y = 3 * x ** 3
    print('x =', x, 'y =', y)
```

[3] *Дописано.* Условия в рукописи нет, оно восстановлено по программе.



Пример 4.4. (к лаб. 4.3). Определить, попадает ли точка (x, y) в заштрихованную область:



Подсказка: отдельно записать условие для $x < 0$, где $x^2 + y^2 \le R^2$.

5. Многоальтернативное ветвление

В Питоне `if`, вложенный в ветку `else`, функционально заменяется на `elif`.

Синтаксис [4] (`[ ]` — часть, которая может отсутствовать):

```
if <выражение 1>:
    <команды 1>
[elif <выражение 2>:
    <команды 2>]
...
[else:
    <команды N+1>]
```

[4] *Исправлено.* В рукописи после первой строки пропущены `<команды 1>`.

Пример 5.1. (к лаб. 5). Соотношение между мерами длины.

Переменные: `k` — код пункта меню.

```
print('1 - миля')
print('2 - верста\n3 - сажень')
print('''4 - аршин
5 - фут''')
k = int(input('Вв. код пункта меню: '))
if k == 1:
    print('1,61 км')
elif k == 2:
    print('1,07 км')
elif k == 3:
    print('2,13 м')
elif k == 4:
    print('0,71 м')
elif k == 5:
    print('0,30 м')
else:
    print('Нет такого пункта меню')
```

В рукописи после ветки `k == 2` стоит многоточие; ветки для `k = 3, 4, 5` и `else` дописаны[5].

5.1. Оператор выбора `match`

Синтаксис:

```
match <выражение-селектор>:
    case <шаблон 1>:
        <команды 1>
    ...
    case <шаблон N>:
        <команды N>
    [case _:
        <команды N+1>]
```

`case _` — ветка «иначе»: выполняется, если не подошёл ни один шаблон. Несколько шаблонов в одной ветке перечисляют через `|`.

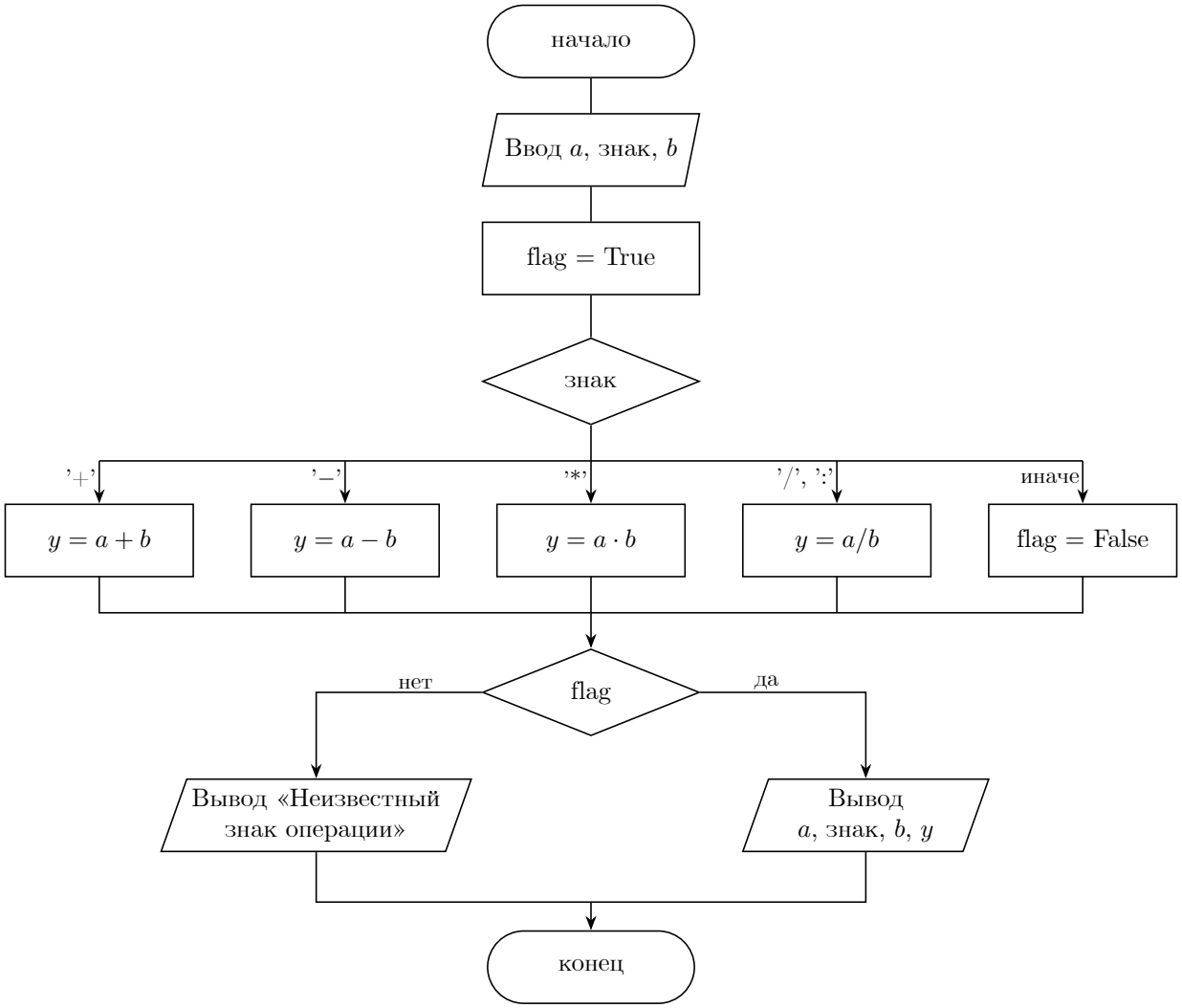
Пример 5.3. Простейший калькулятор.

```
# Переменные:
#     a, b - операнды
#     y - результат
#     znak - знак операции
#     flag - признак существования знака операции
print('?')
a = float(input())
znak = input()
b = float(input())
flag = True
match znak:
    case '+':
        y = a + b
    case '-':
        y = a - b
    case '*':
        y = a * b
    case '/' | ':':
        y = a / b
    case _:
```

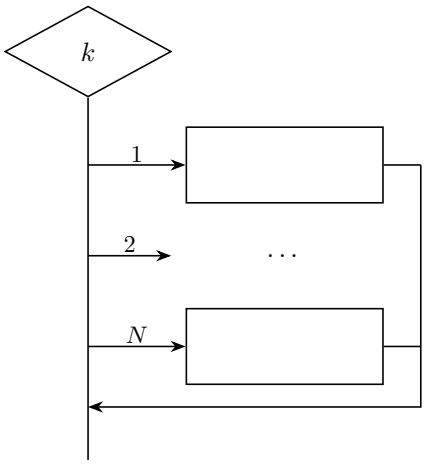
[5] *Дописано.* 1 сажень = 2,1336 м, 1 аршин = 0,7112 м, 1 фут = 0,3048 м.

```
        flag = False
if flag:
    print(a, znak, b, '=', y)
else:
    print('Неизвестный знак операции')
```

Последние четыре строки (вывод результата) дописаны: в рукописи программа и схема обрываются после `match`[6].



Общий вид выбора на схеме. От блока «решение» идёт одна линия, от неё ответвляются выходы, подписанные значениями селектора; после блоков ветви сходятся:



[6] *Дописано.* Добавлены вывод результата и блок «flag» на схеме.

6. Циклы с параметром (со счётчиком) `for`

Когда одни и те же действия необходимо выполнить многократно, для этого используют различные операторы циклов. В программировании различают следующие **типы циклов**:

1) циклы с предусловием;
2) циклы с постусловием;
3) циклы с параметром;
4) циклы с перебором коллекций.

Каждый из перечисленных циклов имеет **общие структурные особенности** и состоит из 4 частей:

1) *подготовка цикла* — необходима для задания начальных значений;
2) *тело цикла* — те команды, которые расположены внутри цикла и выполняются многократно;
3) *модификация* — изменение параметра цикла;
4) *проверка условия* окончания или продолжения цикла.

Синтаксис цикла с параметром:

```
for <параметр> in <список значений>:
    <команды>
```

Как работает: параметр поочерёдно принимает значения из списка, при этом каждый раз выполняются команды из тела цикла.

6.1. Диапазон `range()`

Функция `range()` возвращает диапазон целых чисел от заданного начального значения до конечного с заданным шагом:

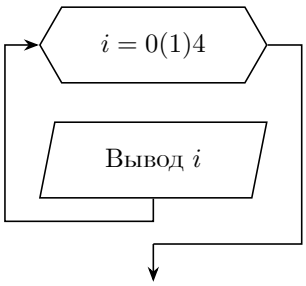
`range(<нач. знач.>, <след. за конечным знач.>, <шаг>)`

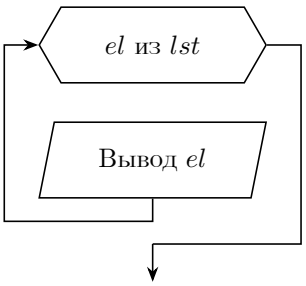
Если начальное значение не указано, то оно = 0; если не указан шаг, то он = +1.

```
for i in range(5):
    print(i)
```

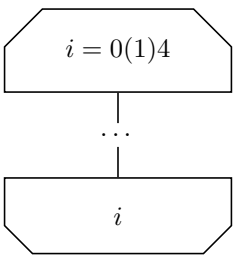
```
lst = [1, 2, 3]
for el in lst:
    print(el)
```

Схема цикла по старому ГОСТу — блок «модификация» (шестиугольник), линия возврата приходит в его левую вершину, выход из цикла — из правой:





По новому ГОСТу цикл изображается двумя блоками: начало цикла (с законом изменения параметра) и конец цикла (с именем параметра):



Пример 6.1. Вывести $n_{\max}$ членов последовательности $a_1 = 2,\ \ a_n = n \cdot a_{n-1}$[7].

```
# Переменные:
#     nmax - количество чисел
#     n - параметр цикла
#     a1, a2 - предыдущее и текущее значения
nmax = int(input('Вв. кол-во чисел\n'))
a1 = 2
print('a1 = 2')
for n in range(2, nmax + 1):
    a2 = n * a1
    print('a%d = %d' % (n, a2))
    a1 = a2
```

Последняя строка `a1 = a2` добавлена[8].

Пример 6.2. Числа Фибоначчи: $f_n = f_{n-1} + f_{n-2},\ \ f_1 = 1,\ \ f_2 = 1.$

```
# Пример 6_2
# Числа Фибоначчи
# Переменные:
#     n - параметр цикла
#     nmax - количество чисел
#     f1, f2, f3 - предыдущие и текущее числа Фибоначчи
nmax = int(input('Вв. количество чисел: '))
f1 = 1
f2 = 1
print('f1 = 1\nf2 = 1')
for n in range(3, nmax + 1):
    f3 = f1 + f2
    print('f%d = %d' % (n, f3))
    f1 = f2
    f2 = f3
```

[7] *Дописано.* Условия в рукописи нет, оно восстановлено по программе.
[8] *Исправлено.* В рукописи нет `a1 = a2`: тогда `a1` всегда равно 2 и вместо $a_n = n\,a_{n-1}$ печатается $2n$.

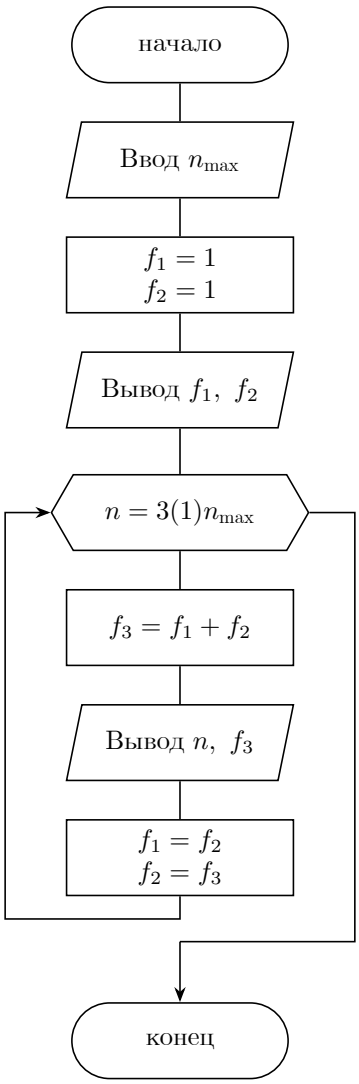


Схема из рукописи[9].

6.2. Форматный вывод: оператор %

В строке ставят спецификаторы, а после строки через % перечисляют значения: `'f%d = %d' % (n, f3)`.

```
# %d - целое число
# %f - вещественное число в формате с фиксированной точкой
print('%f' % 1234568.12)     # 1234568.120000
print('%f' % 1e-3)           # 0.001000
# %e - вещественное число в формате с плавающей точкой
print('%e' % 1234568.12)     # 1.234568e+06
# %s - для вывода строк
print('%10d' % 123)          #        123   (поле из 10 позиций)
print('%8.2f' % 123.4567)    #   123.46     (8 позиций, 2 знака после точки)
print('%.3f' % 123.45678)    # 123.457
print('%20f' % 123.45678)    #           123.456780   (строка из 20 символов)
# %o - вывод числа в 8 СС
print('%o' % 123)            # 173
# %x - вывод целого в 16 СС
print('%x' % 123)            # 7b
```

[9] *Исправлено.* В рукописи после блока $f_3 = f_1 + f_2$ стоит ещё блок ввода-вывода с той же формулой; он лишний и убран.

6.3. Накопление суммы

$$S = f_1 + f_2 + \cdots + f_n = \sum_{i=1}^{n} f_i.$$

Для вычисления такой суммы в программировании можно воспользоваться *рекуррентной формулой*:

$$S_i = S_{i-1} + f_i, \qquad i = 1(1)n = \overline{1,n} \ \ (i = 1, 2, \ldots, n), \qquad S_0 = 0.$$

! Для того чтобы не испортить значение суммы, её начальное значение нужно задать равным нулю ($S_0 = 0$).

Пример 6.3. $S = \sum\limits_{i=1}^{n} \dfrac{1}{i^2} = \dfrac{1}{1} + \dfrac{1}{2^2} + \cdots + \dfrac{1}{n^2}.$

```
# Пример вычисления суммы
# Переменные:
#     i - параметр цикла
#     S - сумма
#     n - количество слагаемых
n = int(input("Вв. количество слагаемых: "))
S = 0
for i in range(1, n + 1):
    S = S + 1 / i ** 2      # или S += 1 / i ** 2
print('Сумма %d слагаемых = %f' % (n, S))
```

6.4. Произведение

$$P = f_1 \cdot f_2 \cdot \ldots \cdot f_n = \prod_{i=1}^{n} f_i, \qquad P_i = P_{i-1} \cdot f_i, \quad i = \overline{1,n}, \qquad P_0 = 1.$$

Пример 6.4. Натуральная степень действительного числа через произведение:

$$a^n = \underbrace{a \cdot a \cdot \ldots \cdot a}_{n \text{ раз}} = \prod_{i=1}^{n} a.$$

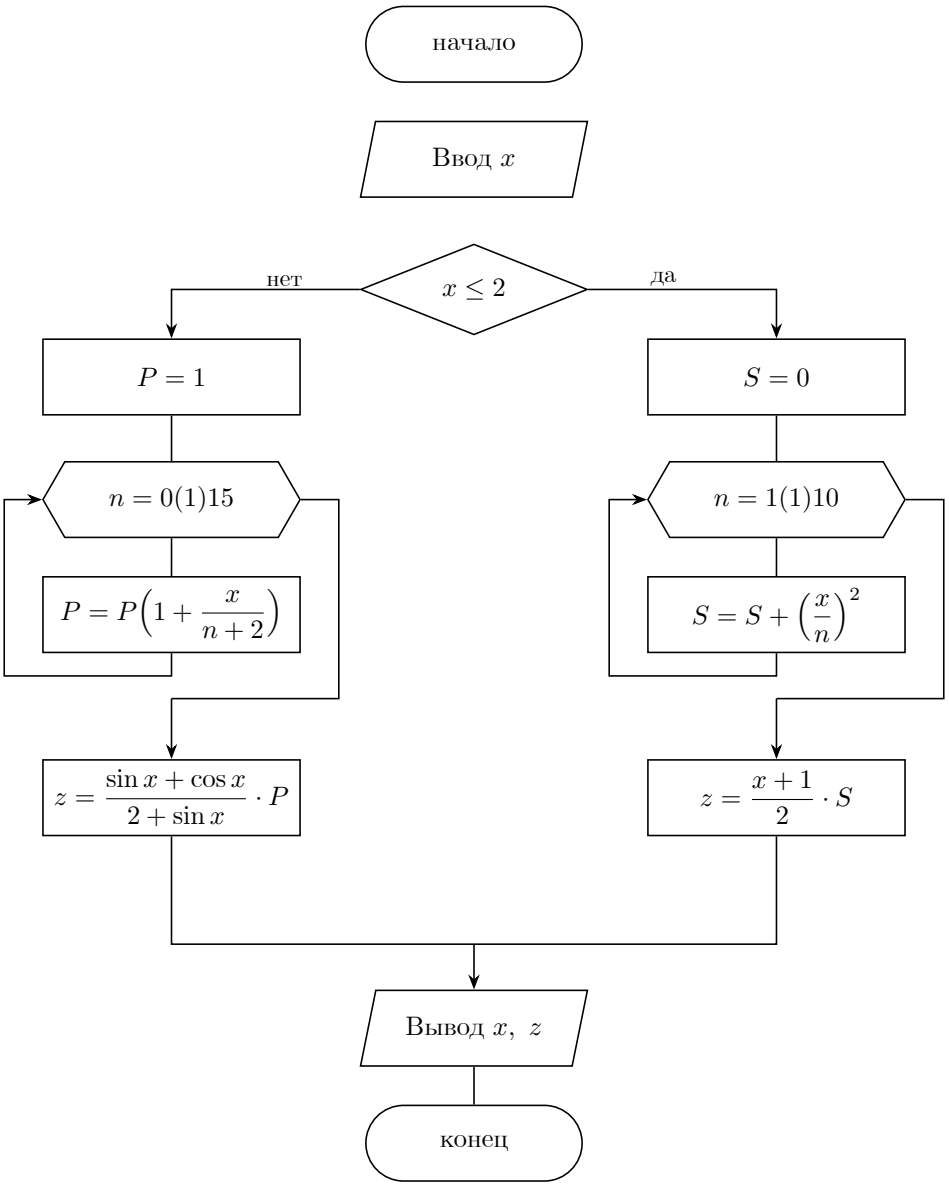
```
# Пример 6_4
# Вычисление через произведение
# натуральной степени заданного действительного числа
# Переменные:
#     P - произведение (результат)
#     a - основание
#     n - натуральная степень
#     i - параметр цикла
print('Введите основание и степень')
a = float(input())
n = int(input())
P = 1
for i in range(1, n + 1):    # можно range(n): i в теле не используется
    P *= a
print('%.2f в степени %d = %.3f' % (a, n, P))
```

Пример 6.5.

$$z = \begin{cases} \dfrac{x+1}{2} \displaystyle\sum_{n=1}^{10} \left(\frac{x}{n}\right)^2, & x \le 2, \\ \dfrac{\sin x + \cos x}{2 + \sin x} \displaystyle\prod_{n=0}^{15} \left(1 + \frac{x}{n+2}\right), & x > 2. \end{cases}$$

```
# Пример 6_5
# Переменные:
#     x - аргумент
#     z - вычисляемая функция
#     n - параметр цикла
from math import sin, cos
x = float(input('Введите x: '))
if x <= 2:
    S = 0
    for n in range(1, 10 + 1):
        S += (x / n) ** 2
    z = (x + 1) / 2 * S
else:
    P = 1
    for n in range(0, 16):
        P *= 1 + x / (n + 2)
    z = (sin(x) + cos(x)) / (2 + sin(x)) * P
print('x = %5.2f \t z = %5.2f' % (x, z))
```

Показатель степени в сумме — как в условии[10].



[10] *Исправлено.* В рукописи в условии $(x/n)^2$, а в программе `(x/n)**n` и на схеме $(1/n)^n$. Здесь везде как в условии. Если на доске было $(x/n)^n$, поменяй показатель во всех трёх местах.

6.5. Табулирование

Определение. *Протабулировать* функцию $y = f(x)$ — это вычислить значения этой функции при различных значениях аргумента.

Закон изменения аргумента записывают так:

$x = x_0(h_x)x_n$, x_0 — начальное значение, h_x — шаг, x_n — конечное значение аргумента.

Пример. $x = 1(0{,}5)3$, $y = f(x) = \sin x$:

$$x = 1:\ y = \sin 1 = \ldots; \qquad x = 1{,}5:\ y = \sin 1{,}5 = \ldots; \qquad \ldots; \qquad x = 3:\ y = \sin 3 = \ldots$$

Поскольку в Питоне в цикле `for` при использовании функции `range()` можно работать с параметрами цикла только целого типа, необходимо ввести переменную N_x, которая равна числу повторений цикла при указанном законе изменения параметра x:

$$N_x = \left[\frac{x_n - x_0}{h_x}\right] + 1, \qquad [\ ] \text{ — целая часть.}$$

! `for x in range(1, 3 + 0.5, 0.5)` — **нельзя!!!**

Пример 6.6. Протабулировать функцию $y = \sin x$.

```
# Переменные:
#     x - аргумент
#     x0 - начальное значение аргумента x
#     xn - конечное значение аргумента x
#     hx - шаг изменения x
#     y - вычисляемая функция
#     Nx - количество точек в диапазоне
#     i - параметр цикла
from math import *
print('Введите x0(hx)xn')      # лучше подписать каждое значение
x0 = float(input())
hx = float(input())
xn = float(input())
Nx = trunc((xn - x0) / hx + 1e-6) + 1      # 1e-6 - погрешность
x = x0
for i in range(Nx):
    y = sin(x)
    print('x = %10.2f \t y = %10.3f' % (x, y))
    x = x + hx
```

Замечание. 10^{-6} добавляют на случай погрешности вещественной арифметики: например, при $x = 0(0{,}1)0{,}7$ в Питоне `0.7 / 0.1` даёт `6.999999999999999`, и без поправки потерялась бы последняя точка.

7. Цикл с предусловием `while`

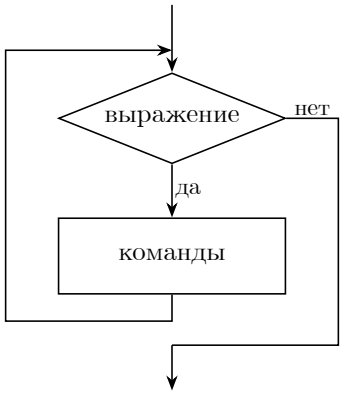
Цикл `for` используется тогда, когда заранее известно число повторений цикла. Когда оно неизвестно, используют циклы с предусловием или с постусловием.

Синтаксис цикла с предусловием:

```
while <выражение>:
    <команды 1>
[else:
    <команды 2>]
```

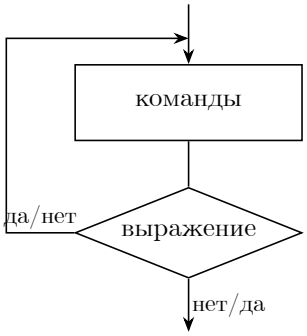
Пока значение выражения истинно, выполняются `<команды 1>` из тела цикла. Как только значение выражения становится ложным, происходит выход из цикла.

! При наличии `else` после *естественного* выхода из цикла (для цикла с предусловием это происходит тогда, когда выражение стало ложно) выполняются `<команды 2>`.



7.1. Цикл с постусловием

Этот цикл отличается от цикла с предусловием тем, что проверка условия продолжения или окончания цикла в нём происходит *после* выполнения тела цикла.



Так как в цикле с предусловием условие проверяется до тела цикла, то, если условие сразу ложно, цикл выполнится 0 раз. **Цикл с постусловием выполнится минимум 1 раз.**

7.2. Общее для циклов `for` и `while`

Оператор `continue`. При выполнении `continue` происходит переход на следующий шаг (итерацию) цикла; при этом на текущей итерации команды, стоящие в теле цикла после `continue`, выполнены не будут.

```
for i in range(1, 6):
    if i == 3:
        continue
    print('2^{} = {}'.format(i, 2 ** i))
```

Оператор `break`. При выполнении `break` в теле цикла происходит досрочный выход из цикла: на текущей итерации не выполняются команды тела цикла, стоящие после `break`, и не выполняются следующие итерации.

```
for i in range(1, 6):
    if i == 3:
        break
    print(f'2^{i} = {2 ** i}')
```

! `break` и `continue` в лабах не используем (считаем, что их нет).

Конструкция `else`. Ветка `else` выполняется, только если цикл закончился естественно, а не через `break`:

```
lst = [1, 2, 3, 4]
for el in lst:
    if el < 0:
        break
else:
    print('В списке ' + str(lst) + ' нет отрицательных')
```

Пример 7.1. (6.3 по `while`). Протабулировать функцию при $x = x_0(h_x)x_n$:

$$y = \begin{cases} -x + 2, & -1 \le x \le 2, \\ -3, & x < -1; \end{cases}$$ при остальных x функция не определена.

```
# Табулирование циклом с предусловием
# Переменные:
#     x, x0, hx, xn - аргумент и параметры его изменения
#     y - функция
print('Введите x0(hx)xn')
x0 = float(input())
hx = float(input())
xn = float(input())
x = x0
while x <= xn + hx / 3:
    if x > 2:
        print(f'x = {x} функция не опр.')
    else:
        if x < -1:
            y = -3
        else:
            y = -x + 2
        print(f'x = {x} y = {y}')
    x = x + hx
```

Значение при $x < -1$ — как в условии[11]. Добавка `hx / 3` в условии цикла — запас на погрешность, чтобы не потерять точку x_n.

Пример 7.2. $f_1 = 2 + \sin 3x$, $f_2 = \cos\dfrac{x}{3}$. Определить аргумент, при котором расстояние между функциями f_1 и f_2 на заданном интервале максимально.

Расстояние $d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$; при $x_1 = x_2 = x$ $d = \sqrt{(y_1 - y_2)^2} = |y_1 - y_2|$.

```
# Пример 7_2
# Переменные:
#     x, x0, hx, xn - аргумент и его параметры
#     y1, y2 - функции
#     d - расстояние
#     dmax - максимальное расстояние
#     xmax - аргумент, при котором расстояние максимально
from math import sin, cos

def f1(x):
    '''Вычисляет и возвращает значение функции f1
    Формальный параметр: x - аргумент'''
    return 2 + sin(3 * x)

def f2(x):
    '''Вычисляет и возвращает значение функции f2'''
    return cos(x / 3)

print('Введите x0(hx)xn')
x0 = float(input())
```

[11] *Исправлено.* В рукописи в условии -3, а в программе `y = 3`. Если в условии опечатка, верни `y = 3`: при $y = 3$ функция непрерывна в точке $x = -1$.

```
hx = float(input())
xn = float(input())

x = x0
dmax = abs(f1(x0) - f2(x0))
xmax = x0
while x <= xn + hx / 4:
    y1 = f1(x)
    y2 = f2(x)
    d = abs(y1 - y2)
    print(f'{x = }\t f1 = {y1:10.2f} f2 = {y2:.3f} {d = :10.2f}')
    if d > dmax:
        dmax = d
        xmax = x
    x += hx
print(f'макс. расст. между ф. {dmax} в точке {xmax}')
```

Замечание. Начальное значение можно задать и так: `dmax = -inf` (`inf` есть в модуле `math`) или `dmax = float('-inf')`[12].

Замечание. Запись `{x = }` в f-строке печатает и имя, и значение: `x = 0.5`.

8. Вложенные циклы

Определение. *Вложенным* называют цикл, содержащий внутри себя один или несколько других циклов. Циклы, охватывающие другие циклы, называют *внешними*, а остальные — *внутренними*.

1) Правила организации внешнего и внутреннего циклов такие же, как и для простого цикла соответствующего вида.
2) Внутренний цикл должен полностью располагаться в теле внешнего цикла.

Циклы, образующие вложенный цикл, условно разбивают на *уровни* (*ранги*): внешний цикл — уровень 0, первый внутренний цикл — уровень 1 и т. д.

Параметры циклов разных уровней изменяются не одновременно. Сначала все свои значения последовательно принимает параметр самого внутреннего цикла при неизменных значениях параметров внешних для него циклов. После этого на один шаг изменяется параметр предыдущего по рангу цикла, и снова все свои значения принимает параметр цикла последнего уровня вложения. Этот процесс происходит до тех пор, пока параметры циклов всех уровней не примут все свои возможные значения.

```
for i in range(1, 4):
    for j in range(3):
        print(i, j)
```

Если во вложенном цикле реализованы 2 циклические структуры с количеством повторений N_1 и N_2 соответственно, то общее число повторений тела внутреннего цикла равно $N_1 \cdot N_2$ (в примере выше $3 \cdot 3 = 9$ строк).

Пример 8.1. Табулирование функции двух переменных $z = f(x, y)$.

Переменные: `x, x0, xn, hx` — для изменения x; `y, y0, yn, hy` — для изменения y; `z` — вычисляемая функция.

Пусть $z = \sin x \cdot \cos 2y$, $x = x_0(h_x)x_n$, $y = y_0(h_y)y_n$.

```
from math import sin, cos
print('Введите x0(hx)xn')
```

[12] *Исправлено.* В рукописи `dmax = inf`, без минуса. При $+\infty$ условие `d > dmax` не выполнится ни разу, и максимум не найдётся.

```
x0 = float(input())
hx = float(input())
xn = float(input())
print('Введите y0(hy)yn')
y0 = float(input())
hy = float(input())
yn = float(input())

x = x0
while x <= xn + hx / 3:
    y = y0
    while y <= yn + hy / 4:
        z = sin(x) * cos(2 * y)
        print(f'{x = } {y = } {z = }')
        y += hy
    x += hx
```

! Начальное значение параметра внутреннего цикла (`y = y0`) задаётся *внутри* внешнего цикла: перед каждым новым x перебор y начинается заново.

Пример 8.2.

$$y = \frac{x + 1{,}5}{3{,}75} \sum_{k=0}^{9} \prod_{n=5}^{3} (k + n) \cdot x^{k+n}, \qquad x = x_0(h_x)x_n.$$

```
# Пример 8_2
# Переменные:
#     x, x0, xn, hx - параметры изменения x
#     y - вычисляемая функция
#     S, P - сумма и произведение
#     k, n - параметры циклов

# вводим x0, hx, xn   <- это обязательно

x = x0
while x <= xn + hx / 3:
    S = 0
    for k in range(10):
        P = 1
        for n in range(5, 2, -1):
            P *= (k + n) * x ** (k + n)
        S += P
    y = (x + 1.5) / 3.75 * S
    print(f'{x = :10.2f}\t{y = :10.3f}')
    x = x + hx
```

Замечание. Произведение по n от 5 до 3 идёт в обратном порядке: `range(5, 2, -1)` даёт $5, 4, 3$. Сумму обнуляют перед циклом по k, произведение — перед циклом по n, т. е. каждое накопление начинают заново на своём уровне вложения.

Пример 8.3.

$$y = \sum_{i=0}^{3} \prod_{j=5}^{2} \sum_{k=4}^{1} \frac{i + j \cdot k}{i \cdot j + k}.$$

```
# Переменные:
#     i, j, k - параметры циклов
#     P - произведение
#     S1, S2 - суммы
S1 = 0
for i in range(4):
    P = 1
    for j in range(5, 1, -1):
```

```
        S2 = 0
        for k in range(4, 0, -1):
            S2 += (i + j * k) / (i * j + k)
        P *= S2
    S1 += P
print('y =', S1)
```

Скобки в дроби расставлены по формуле[13].

Пример 8.4.

$$y = a\prod_{i=1}^{10}\prod_{j=1}^{i}\frac{i}{j+|a|^j}, \qquad a = a_0(h_a)a_n.$$

```
# Переменные:
#     a, a0, ha, an - параметры переменной a
#     y - функция
#     i, j - параметры циклов
#     P1, P2 - произведения
#     P - произведение для вычисления |a|**j

# ввод a0, ha, an

a = a0
while a <= an + ha / 4:
    P1 = 1
    for i in range(1, 11):
        P2 = 1
        P = 1
        for j in range(1, i + 1):
            P = P * abs(a)
            P2 = P2 * i / (j + P)
        P1 = P1 * P2
    y = a * P1
    print(f'{a = } {y = }')
    a += ha
```

Замечание. Степень $|a|^j$ здесь не вычисляется через `**`, а накапливается произведением: на каждом шаге внутреннего цикла `P` умножается на $|a|$, поэтому при текущем j `P` $= |a|^j$. Для этого `P = 1` задают перед внутренним циклом.

9. Списки (одномерные массивы)

Определение. *Список* в Питоне — это индексированная (упорядоченная) динамическая коллекция, которая состоит из одинаковых или различных элементов.
Динамическая — это значит, что количество элементов может меняться во время выполнения программы.

! *Массив*, в отличие от списка в Питоне, состоит из элементов одного типа.

Создание списка (`list()`):

```
# создание списка
lst1 = [4, 5, 1]
lst2 = [1, 'строка', 3.45]
# создание пустого списка
empt1 = []
empt2 = list()
```

[13] *Исправлено.* В рукописи `S2 += (i + j*k / (i*j + k))`: по приоритету операций это $i + \frac{j\cdot k}{i\cdot j+k}$, а не дробь из условия.

Список — *индексированная и упорядоченная* коллекция: у каждого элемента списка есть порядковый номер (индекс), по которому можно обращаться к элементу. Нумерация с нуля; отрицательный индекс считает с конца.

```
a = [1, 2, 3, 4]
print(a[2])            # 3
print(a[-1])           # 4
print([4, 3, 6][2])    # 6
print(a[99])           # Ошибка! (IndexError)
```

Изменение элемента:

```
a[0] = 99
print(a)               # [99, 2, 3, 4]
```

Метод `append()` — добавление в конец списка:

```
a.append(13)
print(a)               # [99, 2, 3, 4, 13]
```

Метод `extend()` — расширение одного списка элементами другого:

```
a1 = [1, 2, 3]
a2 = [44, 55]
a1.extend(a2)
print(a1)              # [1, 2, 3, 44, 55]
```

```
a = 1
b = 2
c = [a, b]
print(c)               # [1, 2]
b = 4
print(c)               # [1, 2]
```

Замечание. В список попадают *значения* переменных на момент создания; изменение `b` потом на список не влияет.

Длина списка:

```
a = [1, 2, 3]
print(len(a))          # 3
```

Удаление элемента по индексу:

```
del a[1]
print(a)               # [1, 3]
```

Соединение списков (сцепление, сложение):

```
a1 = [1, 2]
a2 = [3, 4]
print(a1 + a2)         # [1, 2, 3, 4]
lst = [1, 2, 3, 4]
lst += [99]
print(lst)             # [1, 2, 3, 4, 99]
```

Повтор элементов списка:

```
a = ['k'] * 3
print(a)               # ['k', 'k', 'k']
```

Принадлежность элемента списку (`in`, `not in`):

```
a = [1, 2, 44]
if 44 in a:
    print('содержится')
if 55 not in a:
    print('не содержится')
```

Во втором условии стоит `not in`[14].

9.1. Перебор элементов списка

1 способ — через индексы:

```
for i in range(len(a)):
    print(a[i], i)
# 1 0
# 2 1
# 44 2
```

2 способ — через перебор элементов коллекции:

```
for el in a:
    print(el)
```

9.2. Ввод списка с клавиатуры

1 способ — начинаем с пустого списка и добавляем элементы:

```
a = []
n = int(input('Вв. количество элементов: '))
print(f'Вв. {n} эл-в')
for i in range(n):
    a.append(int(input(f'a[{i+1}] = ')))
print(a)
```

2 способ — сразу создаём список нужной длины и заполняем по индексам:

```
n = int(input())
print(n)
a = [0] * n            # заполнение массива a нулями
for i in range(n):
    a[i] = int(input())
print(a)
```

9.3. Вывод элементов списка

```
print('Массив:')
for i in range(n):
    print(a[i], end=' ')
print()
```

```
print('Массив:')
for el in a:
    print(el, end='\t')
print()
```

`end=' '` — после элемента печатается пробел, а не перевод строки; пустой `print()` в конце переводит строку.

9.4. Срезы списков

`<список>[<нач. инд.>:<инд. след. за кон.>:<шаг>]`

```
a = [1, 2, 3, 4, 5]
print(a[0:len(a):2])   # [1, 3, 5]
print(a[::2])          # [1, 3, 5]
print(a[::-1])         # [5, 4, 3, 2, 1]
```

То же, что `a[::-1]`, циклом:

```
b = []
for i in range(len(a) - 1, -1, -1):
```

[14] *Исправлено.* В рукописи `if 55 in a: print('не')`. Числа 55 в списке нет, и сообщение не напечатается; судя по его тексту, имелось в виду `not in`.

```
    b.append(a[i])
print(b)                # [5, 4, 3, 2, 1]
```

Срезу можно присвоить новые значения:

```
month = ['январь', 'февраль', 'март', 'апрель', 'май', 'июнь',
         'июль', 'август', 'сентябрь', 'октябрь', 'ноябрь', 'декабрь']
spring = month[2:5]
print(spring)           # ['март', 'апрель', 'май']
month[5:8] = ('June', 'July', 'August')
print(month[4:9])       # ['май', 'June', 'July', 'August', 'сентябрь']
```

```
print(month)            # изменённый массив
a = [1, 2, 3, 4, 5, 6]
del a[::2]
print(a)                # [2, 4, 6]
print(a[1:99])          # [4, 6]   (выход за границы в срезе - не ошибка)
print(a[99:1000])       # []
```

9.5. Генератор списков

```
a1 = [k for k in range(7)]
print(a1)               # [0, 1, 2, 3, 4, 5, 6]
a2 = [k ** 2 for k in range(5)]
print(a2)               # [0, 1, 4, 9, 16]
a3 = [k for k in range(7) if k % 3]
print(a3)               # [1, 2, 4, 5]
import random
a4 = [random.random() for _ in range(5)]
print(a4)               # 5 случайных чисел из [0; 1)
```

Замечание. В `a3` условие `k % 3` истинно, когда остаток не равен нулю, т. е. берутся числа, не делящиеся на 3. Имя `_` ставят вместо параметра цикла, если он в теле не нужен.

9.6. Функция `enumerate()`

`enumerate()` даёт сразу и номер, и значение элемента:

```
a1 = [2, 456, 88, 7]
for index, value in enumerate(a1, start=1):
    print(index, value)
# 1 2
# 2 456
# 3 88
# 4 7
```

Без `start` (`for index, value in enumerate(a1):`) нумерация идёт с нуля.

9.7. Методы списков

`append(x)`	добавляет элемент в конец списка
`extend(lst)`	добавляет в конец все элементы другого списка
`insert(i, a)`	вставляет на i-ю позицию элемент a
`remove(a)`	удаляет первое найденное значение элемента a; ошибка, если такого нет
`pop(i)`	удаляет i-й элемент и возвращает его; без индекса — последний
`index(a)`	возвращает индекс первого элемента, равного a
`count(a)`	возвращает количество элементов, равных a
`sort()`	сортирует список (меняет сам список)

```
lst = [1, 2, 3, 4]
lst.insert(2, 'a')
print(lst)              # [1, 2, 'a', 3, 4]

a = [1, 2, 3, 1]
a.remove(1)
print(a)                # [2, 3, 1]

L = [1, 2, 3]
el = L.pop(1)
print(el, L)            # 2 [1, 3]

L = [1, 2, 3, 1]
i1 = L.index(1)
print(i1)               # 0
i2 = L.index(1, i1 + 1)    # искать, начиная с позиции i1 + 1
print(i2)               # 3
```

```
L = [1, 2, 3, 4, 1, 6, 1]
print(L.count(1))       # 3
# то же «вручную» - сколько раз встречается 4:
k = 0
for el in L:
    if el == 4:
        k += 1
print(k)                # 1
```

```
L = [1, 56, -8, 1000, 2]
L.sort()
print(L)                # [-8, 1, 2, 56, 1000]
# функция sorted() не меняет исходный список
L = [4, 7, 1]
print(sorted(L, reverse=True))     # [7, 4, 1]
print(L)                           # [4, 7, 1]
L = [-10, 1, -5, 2, -3]
print(sorted(L, key=abs))          # [1, 2, -3, -5, -10]
```

`reverse=True` — по убыванию; `key=abs` — сравнивать элементы по модулю.

```
L = [1, 2, 3]
L.reverse()             # reverse() - разворачивает список
print(L)                # [3, 2, 1]
print(L[::-1])          # [1, 2, 3]
```

Метод `copy()` — поверхностная копия списка:

```
L1 = [1, 2, 3]
L2 = L1.copy()          # сравни: L2 = L1
L1.reverse()
print(L2, id(L1), id(L2))     # [1, 2, 3], id(L1) != id(L2)
```

Разворачивается список `L1`[15].

При `L2 = L1` будет `id(L1) == id(L2)`. Поэтому при использовании `copy()` значение нового списка не будет изменяться при изменении исходного ($id_1 \neq id_2$). Если списки присвоить через `=` (присваивание), то значение копии будет изменяться при изменении оригинала ($id_1 = id_2$): обе переменные указывают на один и тот же список. Копию можно сделать и срезом: `L3 = L1[:]`.

Метод `clear()` — очистка списка:

```
print(id(L1))           # id1
L1.clear()
print(L1, id(L1))       # [] id2:  id1 == id2 - тот же список
```

[15] *Исправлено.* В рукописи `L1 = reverse()` — такой функции нет, нужен метод `L1.reverse()`.

```
L1 = []
print(L1, id(L1))      # [] id3:  id1 != id3 - новый список
del L1[:]              # тоже очищает список на месте
print(id(L1))
```

Пример 9.1. Нахождение среднего значения элементов одномерного массива.

```
# Переменные:
#     i - индекс элемента
#     n - размер массива
#     a - одномерный массив
#     S - сумма
#     Sr - среднее
n = int(input('Введите размер мас.: '))

a = []
print(f'Введите {n} эл-в массива')
for i in range(n):
    print(f'a[{i+1}] = ', end='')
    a.append(float(input()))
# вывод массива
print('массив:')
for el in a:
    print(el, end=' ')
print()

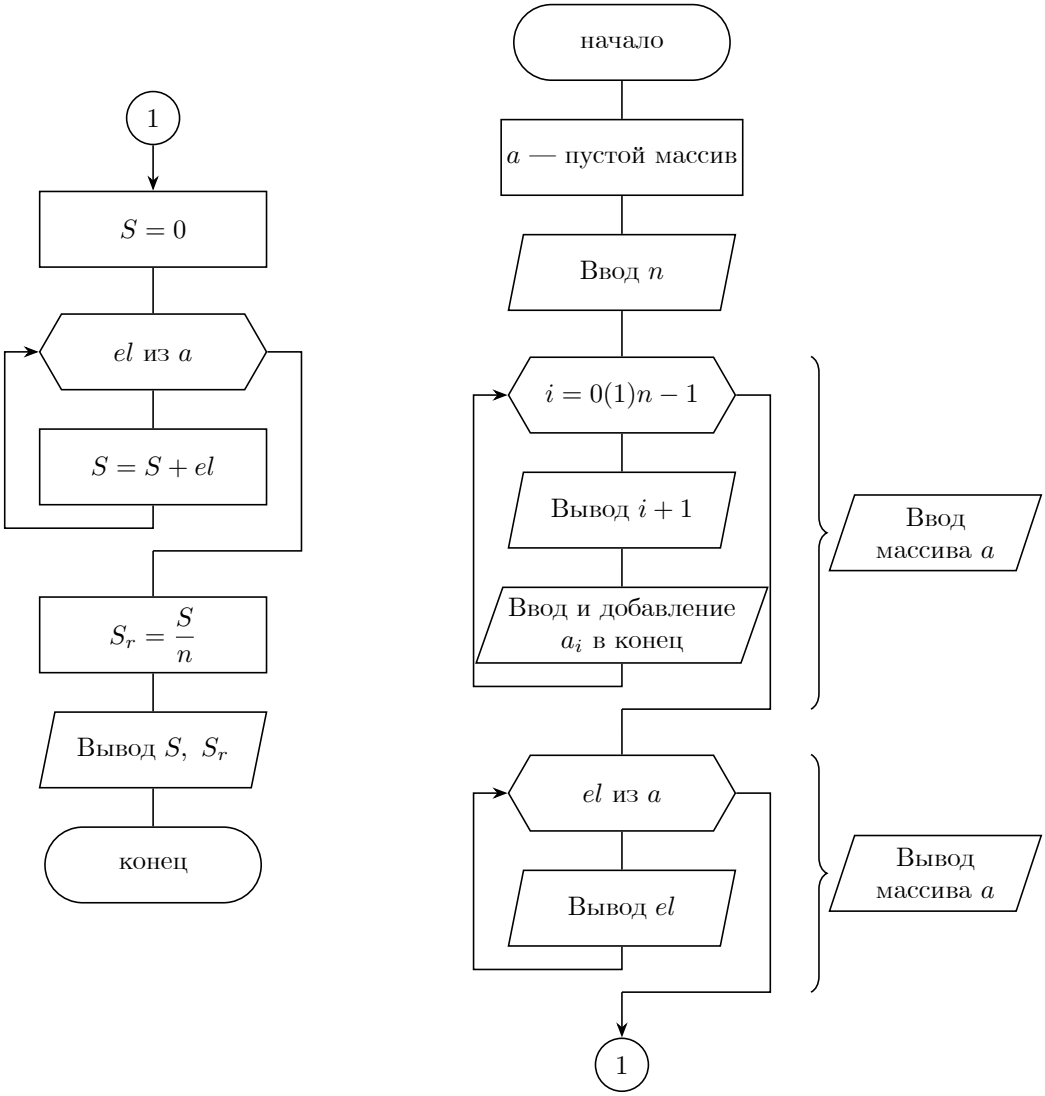
# определение суммы элементов массива
S = 0
for el in a:
    S += el
Sr = S / n
print(f'Сумма элементов массива {S:.2f}')
print(f'Среднее значение {Sr:.2f}')
```

Ввод элементов массива одной строкой (через пробел):

```
lst = list(map(float, input().split()))
print(sum(lst) / len(lst))
```

`input().split()` разбивает строку на части по пробелам, `map(float, …)` переводит каждую часть в число, `list(…)` собирает результат в список.

Схема к примеру 9.1. Справа от цикла — его краткая запись одним блоком; так и рисуют, когда ввод или вывод массива ничем не примечателен.



Пример 9.2. Сортировка массива по убыванию методом пузырька.

```
# Переменные:
#     a - одномерный массив (список)
#     n - размер массива
#     i, k - параметры циклов
#     temp - вспомогательная переменная для обмена элементов
import random

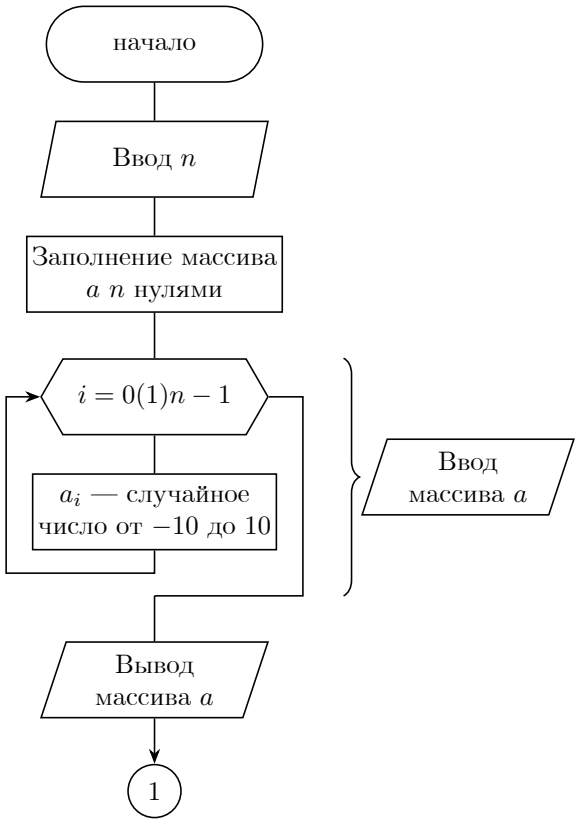
# заполнение массива случайными числами
n = int(input('Вв. кол-во эл-в массива: '))
a = [0] * n
for i in range(n):
    a[i] = random.randint(-10, 10)

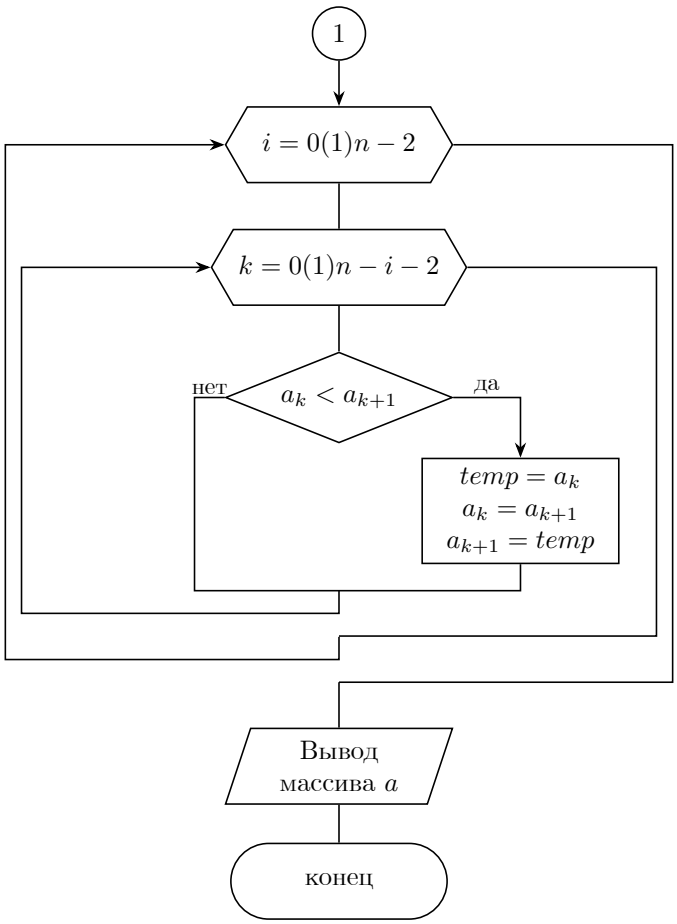
# вывод массива
for i in range(n):
    print(a[i], end='\t')
print()

# сортировка пузырьком
for i in range(n - 1):
    for k in range(n - i - 1):
```

```
        if a[k] < a[k + 1]:          # > - по возрастанию
            temp = a[k]
            a[k] = a[k + 1]
            a[k + 1] = temp
            # или: a[k], a[k + 1] = a[k + 1], a[k]
print('Отсортированный массив:')
print(*a, sep='\t')
```

Замечание. За один проход внутреннего цикла самый маленький из ещё не упорядоченных элементов «всплывает» в конец, поэтому каждый следующий проход на один элемент короче: $k = 0(1)n - i - 2$. Звёздочка в `print(*a, sep='\t')` передаёт элементы списка в `print` по отдельности.





Пример 9.3. Поиск максимума и минимума в массиве.

```
# Переменные:
#     a - массив
#     n - размер массива
#     el - элемент массива
#     i - индекс элемента массива
#     amax - макс. элемент
#     amin - мин. элемент
#     imin - его индекс
a = list(map(int, input('Введите элементы мас.: ').split()))
print(*a)

amax = -float('inf')
for el in a:
    if el > amax:
        amax = el
print('Макс. элемент', amax)

# поиск минимума и его индекса
amin = a[0]
imin = 0
n = len(a)
for i in range(1, n):
    if a[i] < amin:
        amin = a[i]
        imin = i
print(f'min элемент {amin}, его номер {imin + 1}')
```

Замечание. Максимум можно начинать с $-\infty$ (`-float('inf')`): любой элемент больше, и первое же сравнение заменит его. Минимум здесь начинают с первого элемента, тогда цикл идёт с $i = 1$. Номер для человека — `imin + 1`, т. к. индексы считаются с нуля.

10. Матрицы (двумерные массивы)

В рукописи матрицы записаны после кортежей (тема 11), здесь они стоят по номеру темы. У заголовка пометка «без п/п»: примеры этой темы написаны без подпрограмм.

Определение. Если число индексов в описании массива равно 2, то массив называется *двумерным массивом* или *матрицей*.

Матрицу в Питоне можно представить в виде вложенного списка, при этом для обращения к элементу нужно указывать 2 индекса, каждый в своих скобках `[ ]`.

$$A = \begin{pmatrix} 11 & 12 & 13 & 14 \\ 21 & 22 & 23 & 24 \\ 31 & 32 & 33 & 34 \end{pmatrix} \quad \begin{array}{l} \text{строки — индекс } i\text{:} \\ i = 0, 1, \ldots, n-1 \end{array}$$

$$\text{столбцы — индекс } j\text{: } j = 0, 1, \ldots, m-1$$

$A_{n\times m}$: n — число строк, m — число столбцов.

```
A = [[11, 12, 13, 14],
     [21, 22, 23, 24],
     [31, 32, 33, 34]]
# A[i][j] - элемент из i-й строки и j-го столбца
print(A[1][2])         # 23
```

10.1. Создание и ввод матрицы

Создание матрицы $A_{n\times m}$ через генератор и заполнение её нулями:

```
A = [[0] * m for _ in range(n)]
```

! Не пиши `A = [[0] * m] * n`: так получится n ссылок на *одну и ту же* строку, и присваивание `A[0][0] = 5` изменит первый элемент во всех строках.

Ввод элементов матрицы:

```
n = int(input('Вв. количество строк: '))
m = int(input('Вв. количество столбцов: '))
A = [[0] * m for _ in range(n)]
print(f'Введите {n * m} элементов матрицы')
for i in range(n):
    for j in range(m):
        A[i][j] = float(input())
print(A)
```

Ввод по строкам: каждая строка матрицы вводится одной строкой через пробел

```
B = []
for i in range(n):
    row = [int(el) for el in input().split()]
    B.append(row)
print(B)
```

или по одному элементу:

```
B = []
for i in range(n):
    row = []
    for j in range(m):
        row.append(int(input()))
    B.append(row)
print(B)
```

10.2. Вывод матрицы

```
for i in range(n):
    for j in range(m):
        print(A[i][j], end='\t')
    print()
```

```
for row in A:
    for el in row:
        print(el, end='\t')
    print()
```

10.3. Пример

Пример 10.1. Найти в матрице максимальный элемент, минимальный элемент и его индексы. Заменить минимальный элемент максимальным.

```
# Переменные:
#     a - матрица
#     i, j - параметры циклов
#     n, m - число строк и столбцов матрицы
#     amin - мин. элемент
#     imin, jmin - его индексы
#     amax - макс. элемент

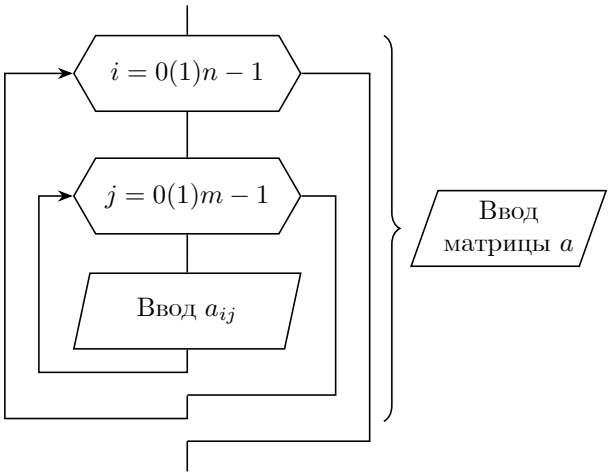
# ввод матрицы (см. п. 10.1)
# вывод матрицы (см. п. 10.2)

# поиск минимума и максимума
amin = a[0][0]
imin = 0
jmin = 0
amax = -float('inf')
for i in range(n):
    for j in range(m):
        if a[i][j] > amax:
            amax = a[i][j]
        if a[i][j] < amin:
            amin = a[i][j]
            imin = i
            jmin = j
print(f'Макс. элемент {amax}')
print(f'Мин. элемент {amin}, его индексы [{imin}][{jmin}]')

# замена минимума на максимум
a[imin][jmin] = amax

# вывод матрицы
print('Матрица после замены:')
for row in a:
    for el in row:
        print(el, end='\t')
    print()
```

На схеме двойной цикл ввода (вывода) матрицы можно заменить одним блоком:



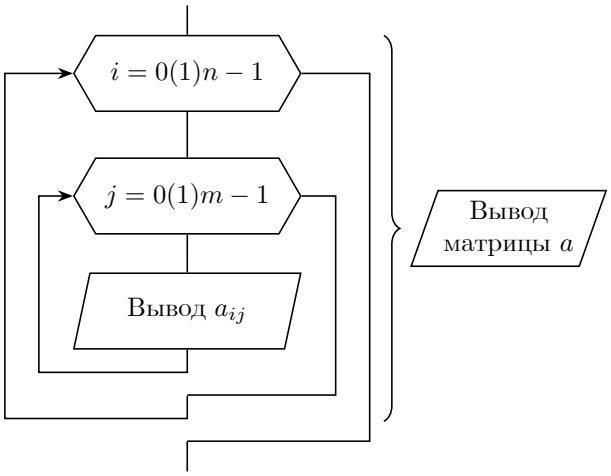
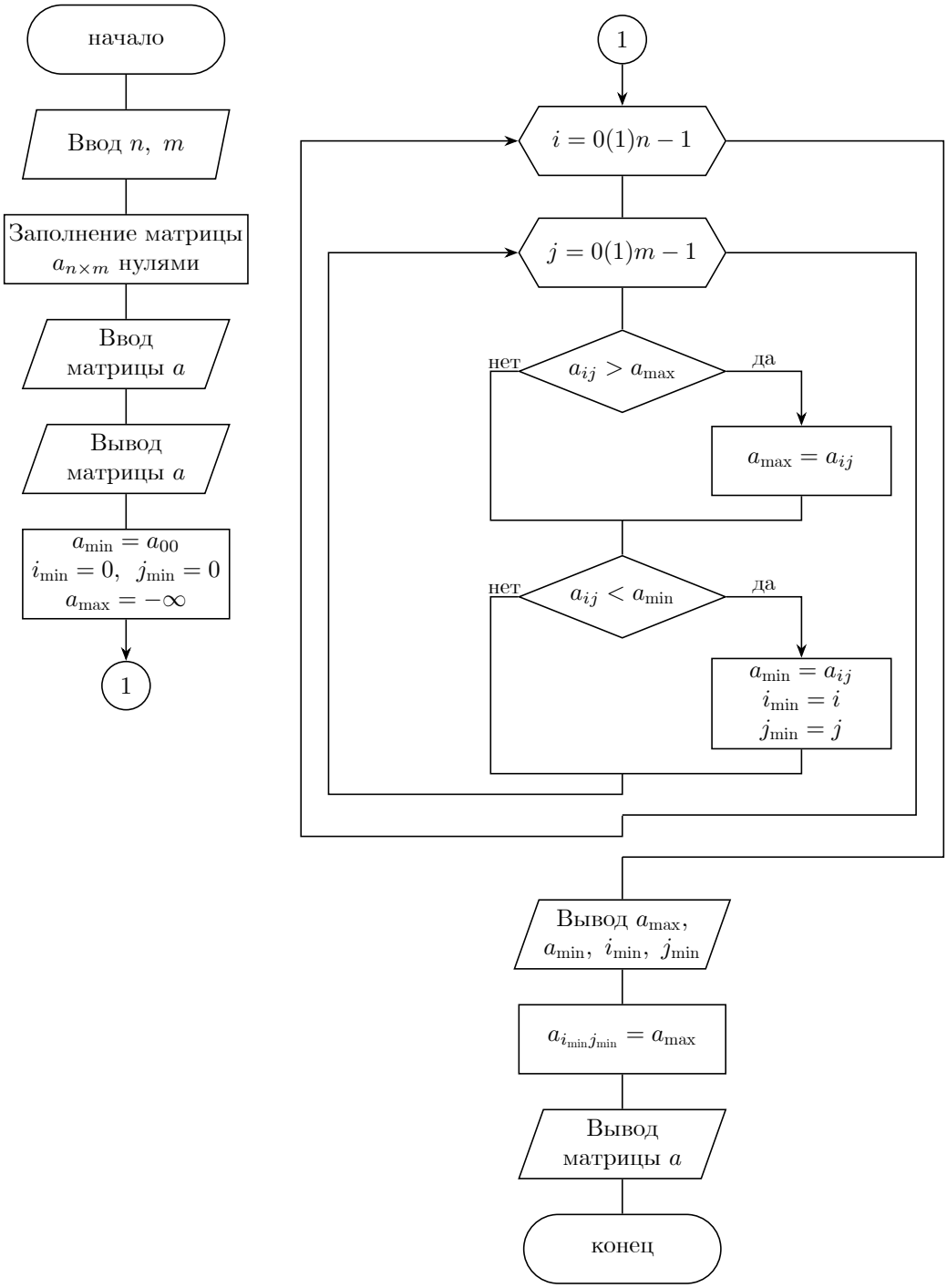


Схема к примеру 10.1[16]



11. Кортежи

Определение. *Кортеж* — это упорядоченная неизменяемая коллекция, используемая для хранения набора элементов. *Упорядоченная* означает, что у каждого элемента есть свой порядковый номер (индекс).

Отличие кортежа от списка состоит в том, что кортежи нельзя изменять.

Ключевое слово, отвечающее за данную коллекцию, — `tuple`. Элементы кортежа заключаются в круглые скобки, типы данных могут быть любыми.

[16] *Дописано.* В рукописи схема не закончена: после блока начальных значений стоят пустые блоки циклов и решения. Здесь она достроена по программе.

```
card = ('7', 'пики')
empty1 = ()             # пустой кортеж
empty2 = tuple()
```

! Для создания кортежа из одного элемента необходимо поставить после этого элемента запятую:

```
t1 = (18,)
t2 = ('one',)
```

Если не возникает неоднозначности, элементы кортежа можно перечислить без скобок:

```
k1 = (13, 22, 444, -5)
k2 = 13, 22, 444, -5
print(k1 == k2)         # True
```

Замечание. В рукописи записано `id(k1) = id(k2)`. В программе так и выходит: Питон хранит два одинаковых кортежа-константы как один объект. Но это особенность интерпретатора, а не правило, поэтому кортежи сравнивают через `==`.

```
print(card[0], len(card), card + t1)
# 7 2 ('7', 'пики', 18)
```

Для кортежей тоже есть операция среза, она возвращает новый кортеж.

К кортежам применимы операции сравнения, сравнение производится поэлементно:

```
(1, 2) == (1, 3)                  # False
(1, 2) < (1, 3)                   # True
(1, 2) < (5,)                     # True
('7', 'червей') < ('7', 'треф')   # False
(1, 2) < ('7', 'пик')             # ошибка: число нельзя сравнить со строкой
```

Для обработки кортежей можно использовать функции и методы списков, кроме тех, которые изменяют элементы.

11.1. Присваивание кортежей

Кортежи можно присваивать друг другу:

```
n, s = 10, 'hello'
a, b = b, a             # обмен значений
```

11.2. Для чего нужны кортежи

1) Набор данных (список), который нужно защитить от изменений.
2) Обмен значений переменных местами.
3) Если в цикле есть набор переменных, которые изменяются одновременно.
4) Кортежи занимают меньше места в памяти.
5) Кортежи работают быстрее.
6) В функции — чтобы вернуть несколько значений:

```
def f(a, b):
    return a + b, a - b

y1, y2 = f(5, 2)
k = f(5, 2)
print(k[0], k[1])      # 7 3
```

7) Для передачи переменного числа аргументов в п/п можно использовать кортеж:

```
def proc(*args):
    print(type(args))  # класс tuple
    for value in args:
        print(value)
```

```
proc(1, 2, 3)
```

8) Кортеж может быть использован в словаре в качестве составного ключа, например координат или ФИО.
9) Кортеж может быть параметром цикла `for`:

```
lst = [11, 2, 43, 4]
for i, el in enumerate(lst, start=1):
    print(i, el)
```

или пример для пункта 8[17]:

```
locations = {(55.76, 37.62): 'Москва', (54.63, 39.74): 'Рязань'}
for coord, city in locations.items():
    print(f'{city} на координатах {coord}')
```

10) В некоторых библиотеках параметры п/п тоже могут быть кортежами:

```
import turtle
turtle.write('Привет!', font=('Arial', 16, 'bold'))
```

11) Для ввода нескольких переменных одной строкой:

```
a, b = map(int, input().split())
```

11.3. Ввод кортежа с клавиатуры

```
n = int(input('Введите число элементов кортежа: '))
k = tuple(int(input(f'Введите число {i + 1}: ')) for i in range(n))
print(k)
```

[17] *Дописано.* Словарь `locations` добавлен, чтобы пример можно было запустить.